AI시대, 차세대 클라우드의 새로운 비전과 기술

openKcloud DevDay 2025

From Chip to Cloud: Kubernetes에서 NPU 인프라 개발

한국전자통신연구원

차재근 선임

목 차

1. 연구과제 개요
2. Kubernetes에서 가속기 관리
3. Kubernetes에서 가속기의 컨테이너 연동

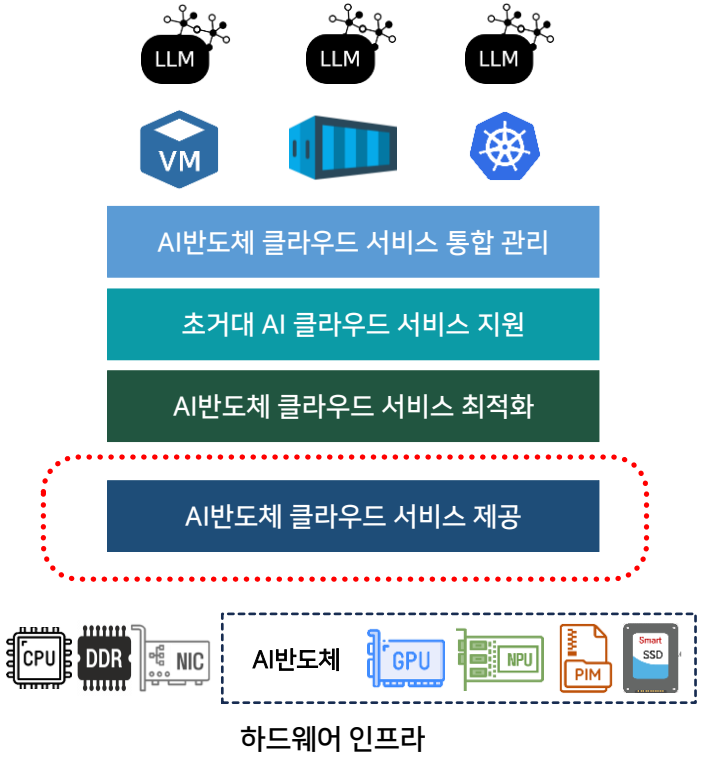
1. 연구과제 개요

연구과제 개요

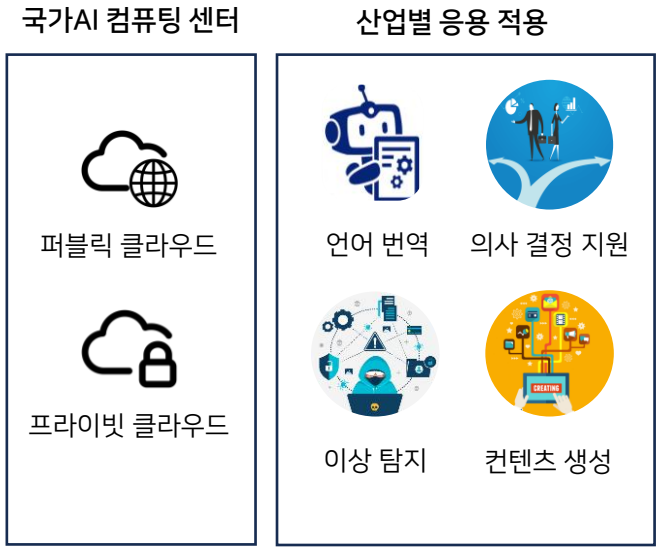
- **목표**
  - 고성능/저전력 AI모델 서비스 최적성능/운영비용 최적화 AI반도체 클라우드 서비스 플랫폼

K-클라우드 사업 결과물 연계

AI반도체 클라우드 서비스 플랫폼

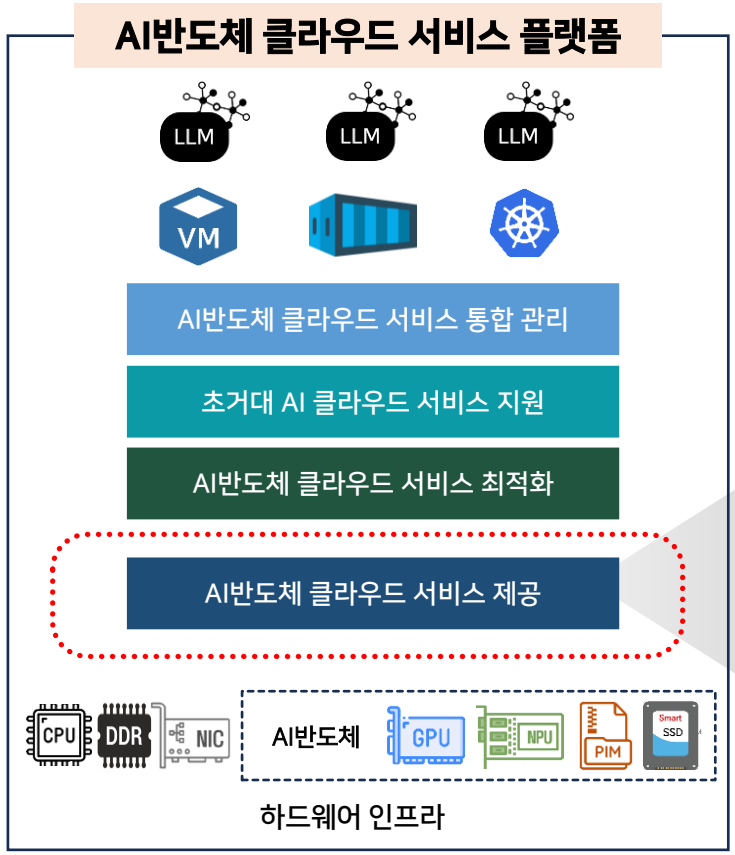


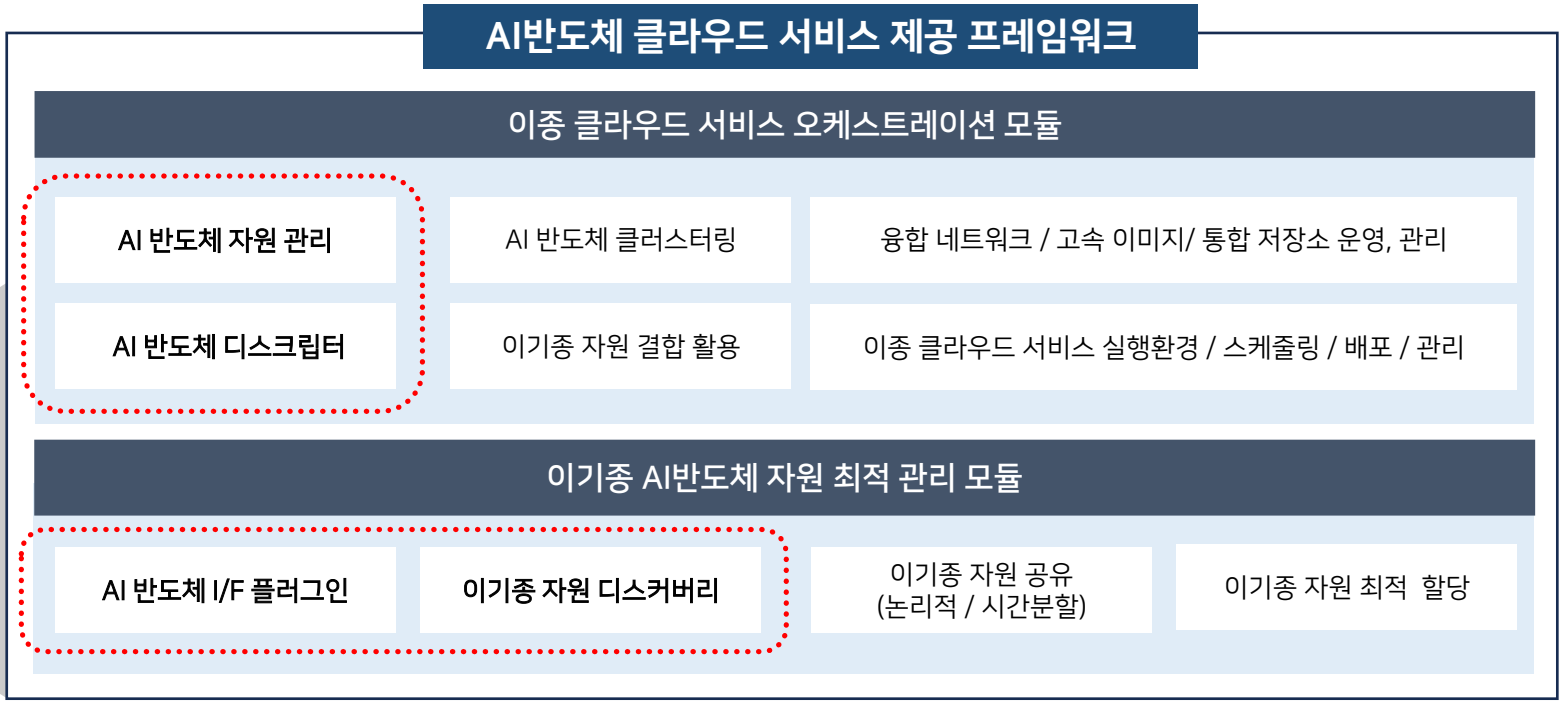
플랫폼 기술의 적용 및 활용



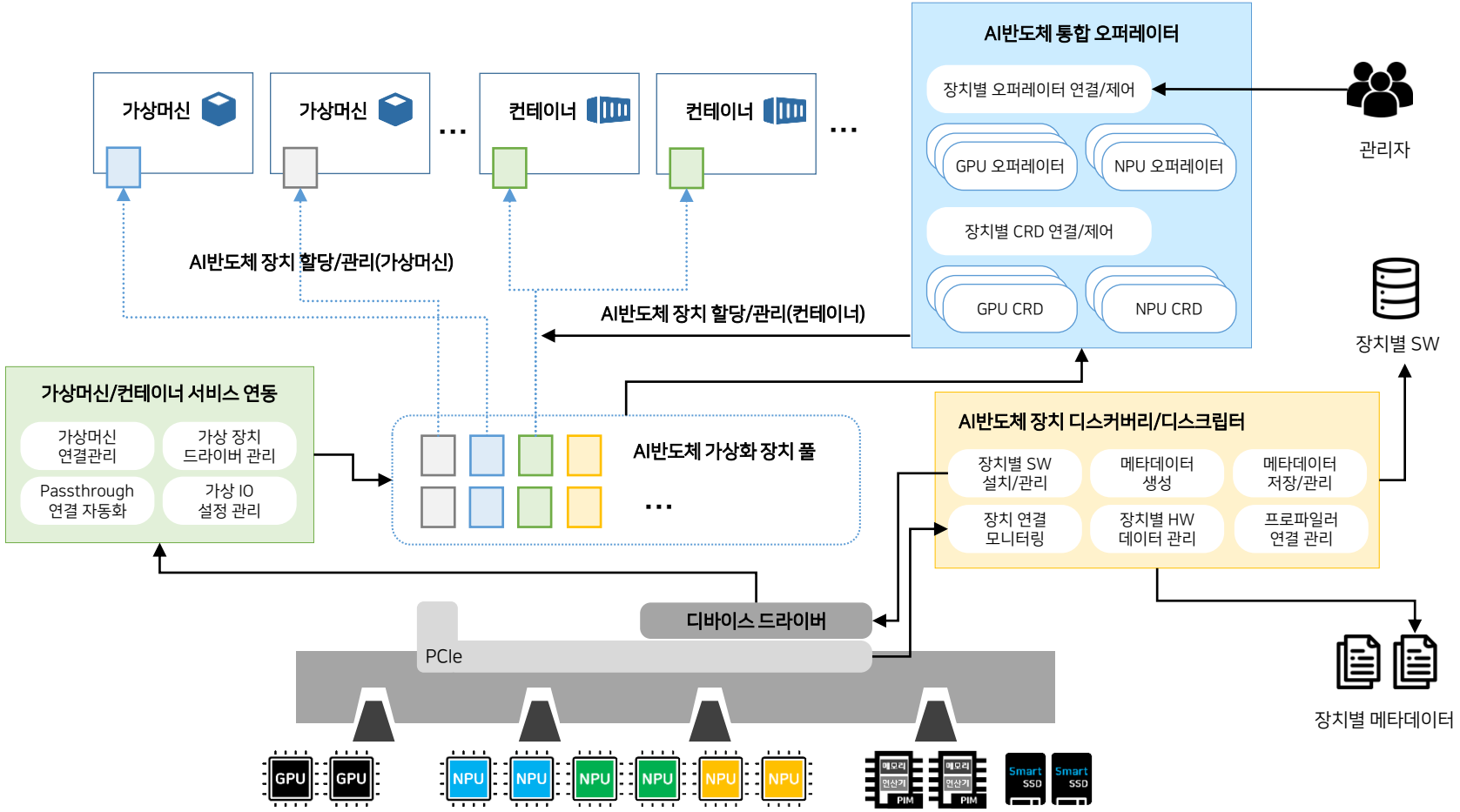
수행 과제 내 위치

- AI반도체 클라우드 서비스 제공 프레임워크





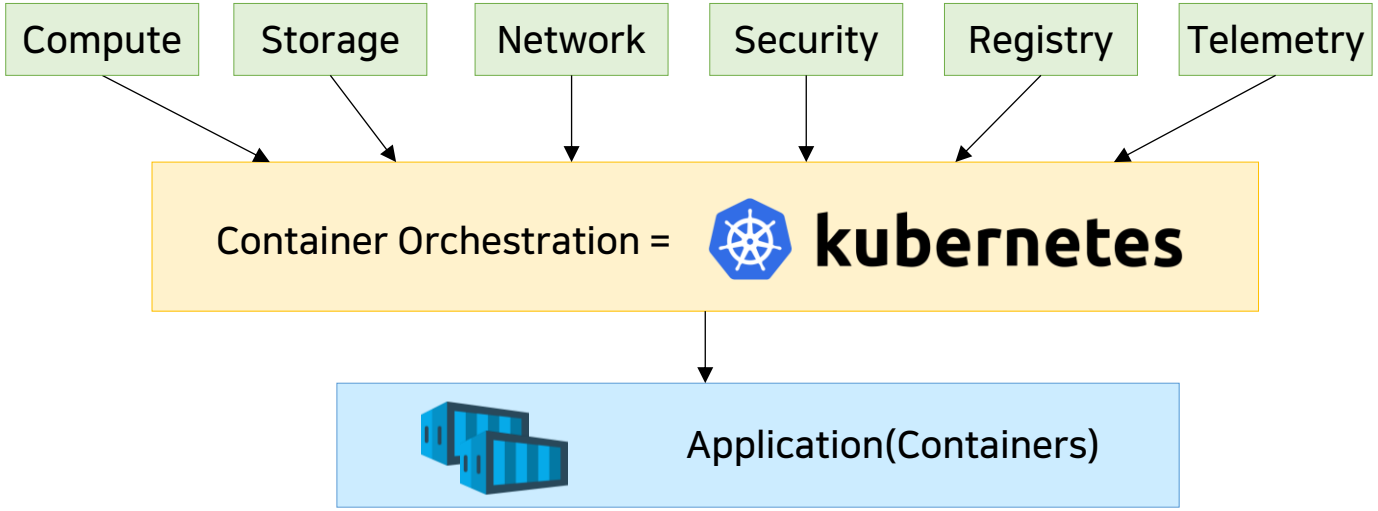
주요 기능 개념도


가상머신
가상머신
컨테이너
컨테이너
AI반도체 통합 오퍼레이터
장치별 오퍼레이터 연결/제어
GPU 오퍼레이터
NPU 오퍼레이터
장치별 CRD 연결/제어
GPU CRD
NPU CRD
관리자
장치별 SW
AI반도체 장치 할당/관리(가상머신)
AI반도체 장치 할당/관리(컨테이너)
가상머신/컨테이너 서비스 연동
가상머신 연결관리
가상 장치 드라이버 관리
Passthrough 연결 자동화
가상 IO 설정 관리
AI반도체 가상화 장치 풀
AI반도체 장치 디스커버리/디스크립터
장치별 SW 설치/관리
메타데이터 생성
메타데이터 저장/관리
장치 연결 모니터링
장치별 HW 데이터 관리
프로파일러 연결 관리
디바이스 드라이버
PCIe
장치별 메타데이터
GPU
NPU
PIM
Smart SSD

2. Kubernetes에서 가속기 관리

Kubernetes 란?

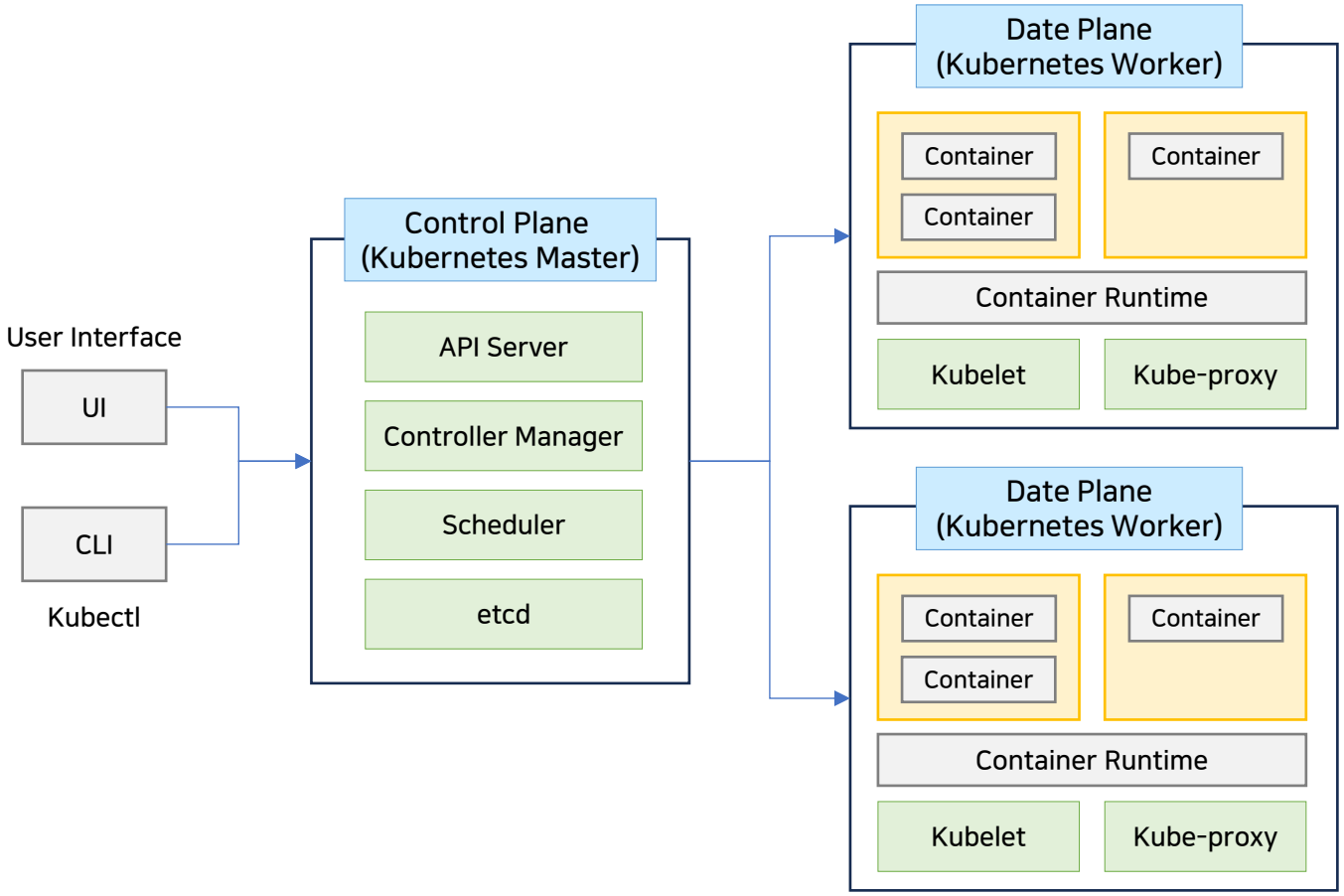
- 컨테이너 오케스트레이션 플랫폼
  - Cloud Native Computing Foundation (CNCF)에서 관리하는 오픈소스
  - 컨테이너화된 애플리케이션의 배포, 확장, 관리 자동화
  - 여러 개의 서버를 클러스터화하여 관리
  - 선언적 구성 기반으로 애플리케이션이 원하는 상태(desired state) 유지



<컨테이너 오케스트레이션 플랫폼 역할>

Kubernetes 구조

- Control Plane (Master)
  - 상태 판단, 명령, 감시
  - 사용자, 시스템간 통신 제공
  - 리소스 배치 결정
  - 실제 상태와 원하는 상태 차이 감지, 조정
  - 클러스터 상태 저장, 관리
- Date Plane (Worker)
  - 컨테이너(Pod) 실행 서버들
  - Control Plane과 통신, 서버 관리
  - 컨테이너 네트워크 트래픽 처리

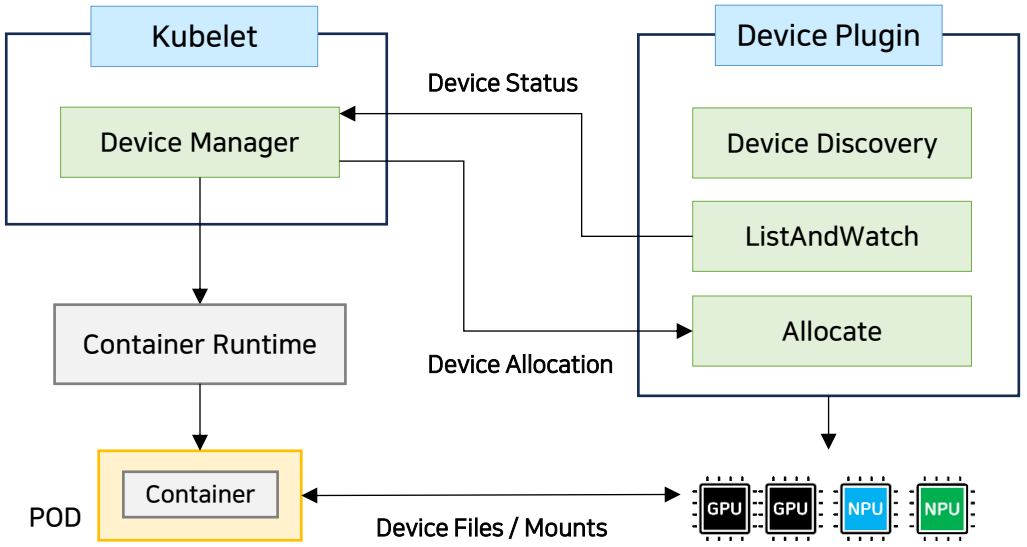


<Kubernetes 구조>

Kubernetes의 가속기 장치 관리

- Device Plugin
  - 특정 하드웨어(GPU, NPU, FPGA ..) 자원을 Kubernetes에 노출, 관리
  - 벤더별 특정 장치를 활용을 위한 확장 플러그인 형태로 배포

  - 동작 과정
    - 노드에서 하드웨어 감지
    - Kubelet과 통신(gRPC)
    - 장치 상태 보고
    - Pod가 장치 요청 시, 장치 할당 (Allocation)
    - 컨테이너에 장치 자원 주입
    - 종료 시, 자원 회수



<Device Plugin 구조>

Device Plugin 한계

- Device Plugin 은 장치 드라이버?
  - 장치 드라이버 = 하드웨어를 OS에서 사용할 수 있게 만드는 것
  - Device Plugin = 이미 OS에서 보이는 장치를 Kubernetes에 알려주는 것

- 사용하려는 장치별로 Device Plugin 배포 전에 ..
  - 장치 드라이버 설치 필요
  - 컨테이너 툴킷 설치 필요
    - 컨테이너 안에서 장치를 사용할 수 있게 하는 것

- 사용하려는 장치와 관리하려는 노드 증가하면?

→

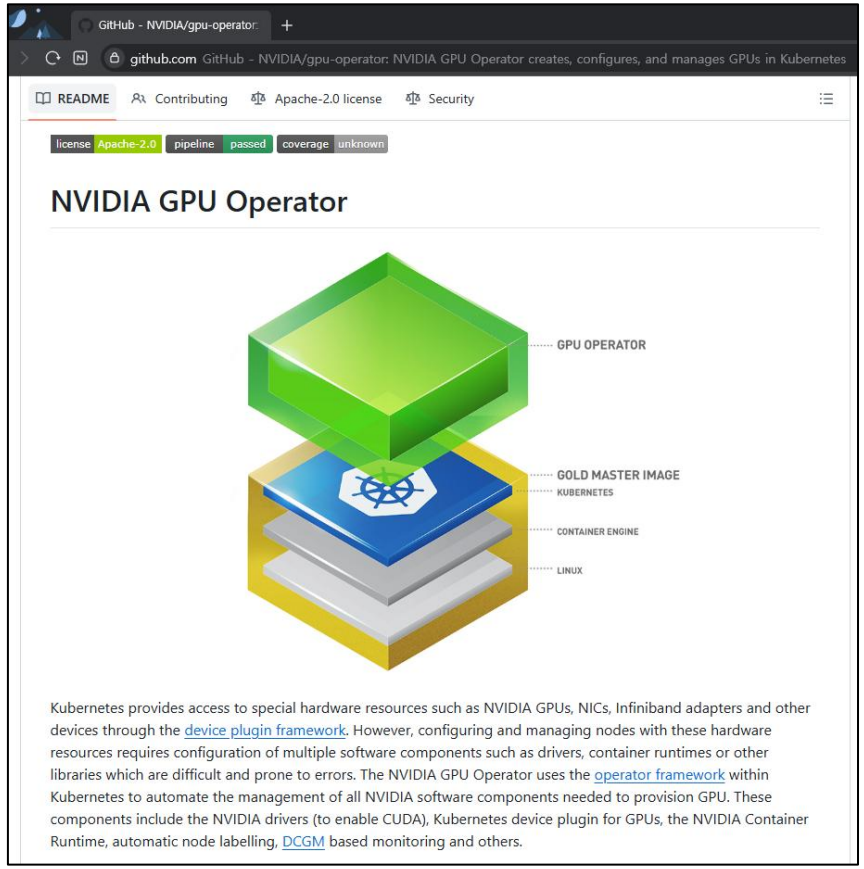
- 노드마다 드라이버 설치, 툴킷 구성 반복
- 벤더, 모델, 버전이 다르면 설치 방식도 달라짐
- 장애 시, 드라이버 문제? 플러그인 문제?

→ 운영 부담이 폭발적으로 증가함

설치를 자동화한다면? (1/2)

- NVIDIA GPU Operator
  - Kubernetes 환경에서 NVIDIA GPU 자원을 자동으로 설치, 구성, 관리하는 도구
  - 기능
    - NVIDIA GPU 드라이버 설치
    - NVIDIA 컨테이너 툴킷 설치, 설정
    - Kubernetes GPU Device Plugin 설치
    - GPU Monitoring 구성 (DCGM Exporter)
    - 자동 롤아웃/업데이트

  - 단, NVIDIA GPU 전용
    - 다른 가속기(AI반도체)는?



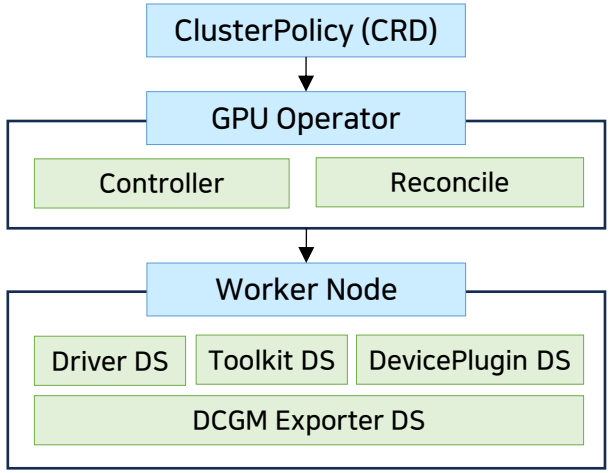
<NVIDIA GPU Operator (Github Repo.)>

설치를 자동화한다면? (2/2)

- NVIDIA GPU Operator
  - 원하는 상태(CRD)를 기준으로 노드에 필요한 구성 요소가 설치되었는지 계속 확인(Reconcile)
  - 모니터링 리소스가 누락 또는 오류가 있으면 다시 생성하여 상태를 유지
  - 구조 및 운영 예
    - CRD (Custom Resource Definition) : 특정 자원 상태의 사용자 요구사항 정의
    - Controller, Reconcile : 상태 감지, 적절한 DaemonSet/Pod 자동 배포

```yaml
apiVersion: nvidia.com/v1
kind: ClusterPolicy
metadata:
  name: cluster-policy
spec:
  driver:
    enabled: true
  toolkit:
    enabled: true
  devicePlugin:
    enabled: true
  dcgmExporter:
    enabled: true
  migManager:
    enabled: false
```

<GPU Operator CRD 예>



※DS (DaemonSet)
※ DCGM (Data Center GPU Manager)

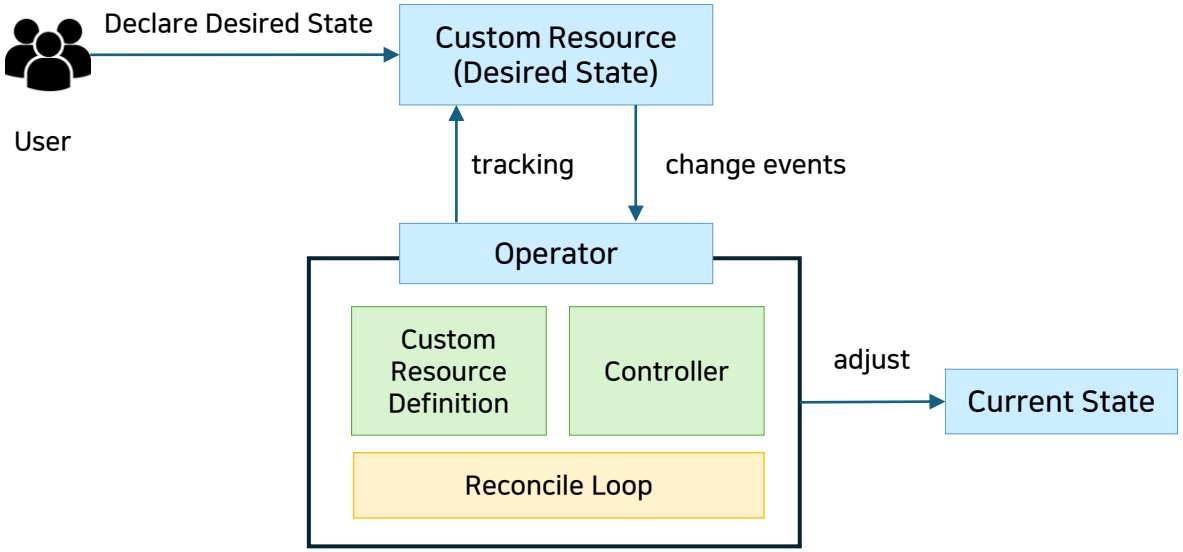
<GPU Operator 구조>

```
kcloud@kcloud-93:~$ kubectl get pod -n gpu-operator -o wide
NAME                                                          READY
gpu-feature-discovery-wx95m                                   1/1
gpu-operator-9b4456d9b-w4tp8                                  1/1
gpu-operator-node-feature-discovery-gc-75ddf784b-pj9pq        1/1
gpu-operator-node-feature-discovery-master-f8c54b974-d2gtm    1/1
gpu-operator-node-feature-discovery-worker-c9ggq              1/1
gpu-operator-node-feature-discovery-worker-ps8f6              1/1
nvidia-container-toolkit-daemonset-6k4br                      1/1
nvidia-cuda-validator-zj97f                                   0/1
nvidia-dcgm-exporter-9ttbn                                    1/1
nvidia-device-plugin-daemonset-8zckx                          1/1
nvidia-driver-daemonset-5hdvq                                 1/1
nvidia-mig-manager-4nf5w                                      1/1
nvidia-operator-validator-xx4sg                               1/1
```

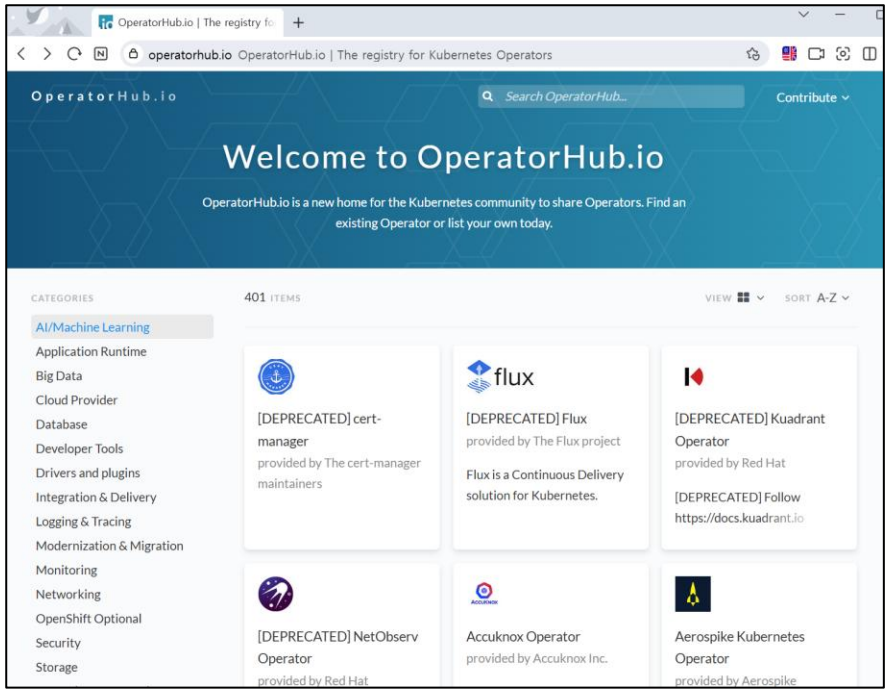
<GPU Operator 배포 결과>

Operator란? (1/1)

- Operator ≒ Controller
  - 클러스터 내 리소스 원하는 상태와 실제 상태를 자동으로 일치시키는 컴포넌트
  - Kubernetes default 리소스 = Controller, 사용자 정의 리소스 = Operator



<Kubernetes Operator(Controller)>

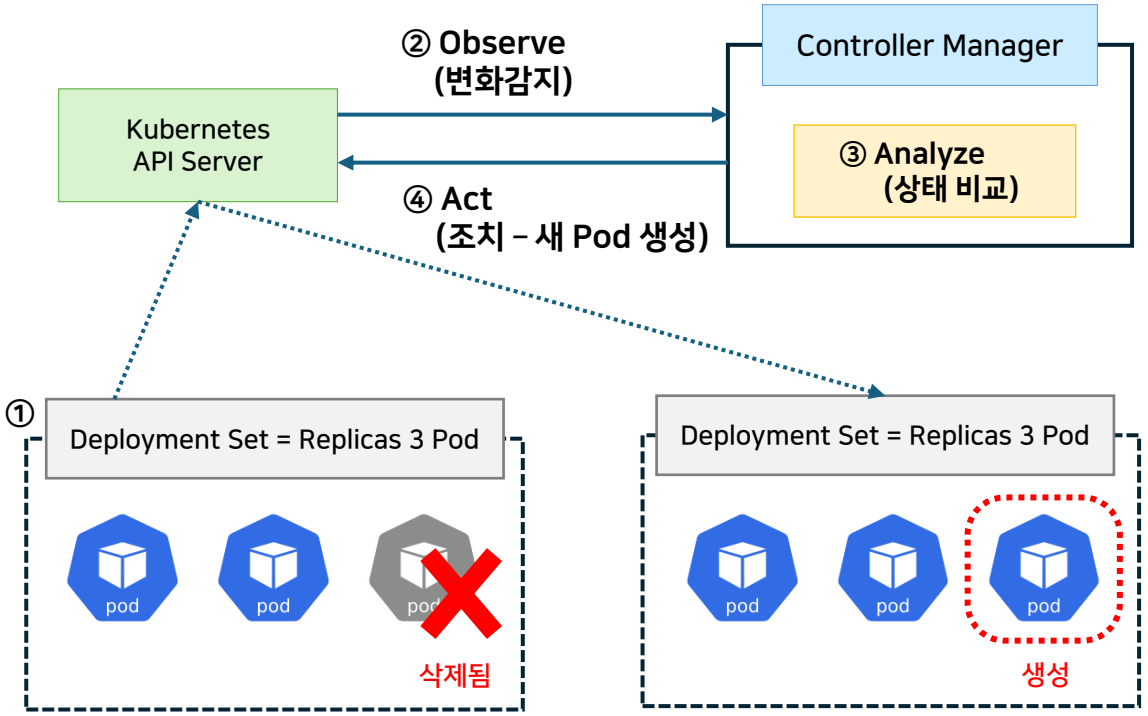


<OperatorHub.io>

Operator란? (1/2)

- Reconcile Loop
  - 변화가 생기면 감지하고, 원하는 상태와 다르면 자동으로 수정 실행

① 배포(Desired State)
클러스터에 항상 3개의 Pod를 유지하도록 설정
② 감시(Observe)
Kubernetes Controller Manager가 해당 Pod들을 주시
③ 상태변경(Current State)
3개의 Pod 중, 1개 Pod가 삭제되면, Current State 변경
④ 복구(Reconcile)
Controller Manager가 상태 비교 후, 부족한 1개 Pod 자동 생성



<Reconcile Loop 동작 과정(Pod 삭제 시, 재생성)>

참조: https://kingofbackend.tistory.com/298

3. Kubernetes에서 가속기의 컨테이너 연동

이기종 가속기를 손쉽게 관리하려면?

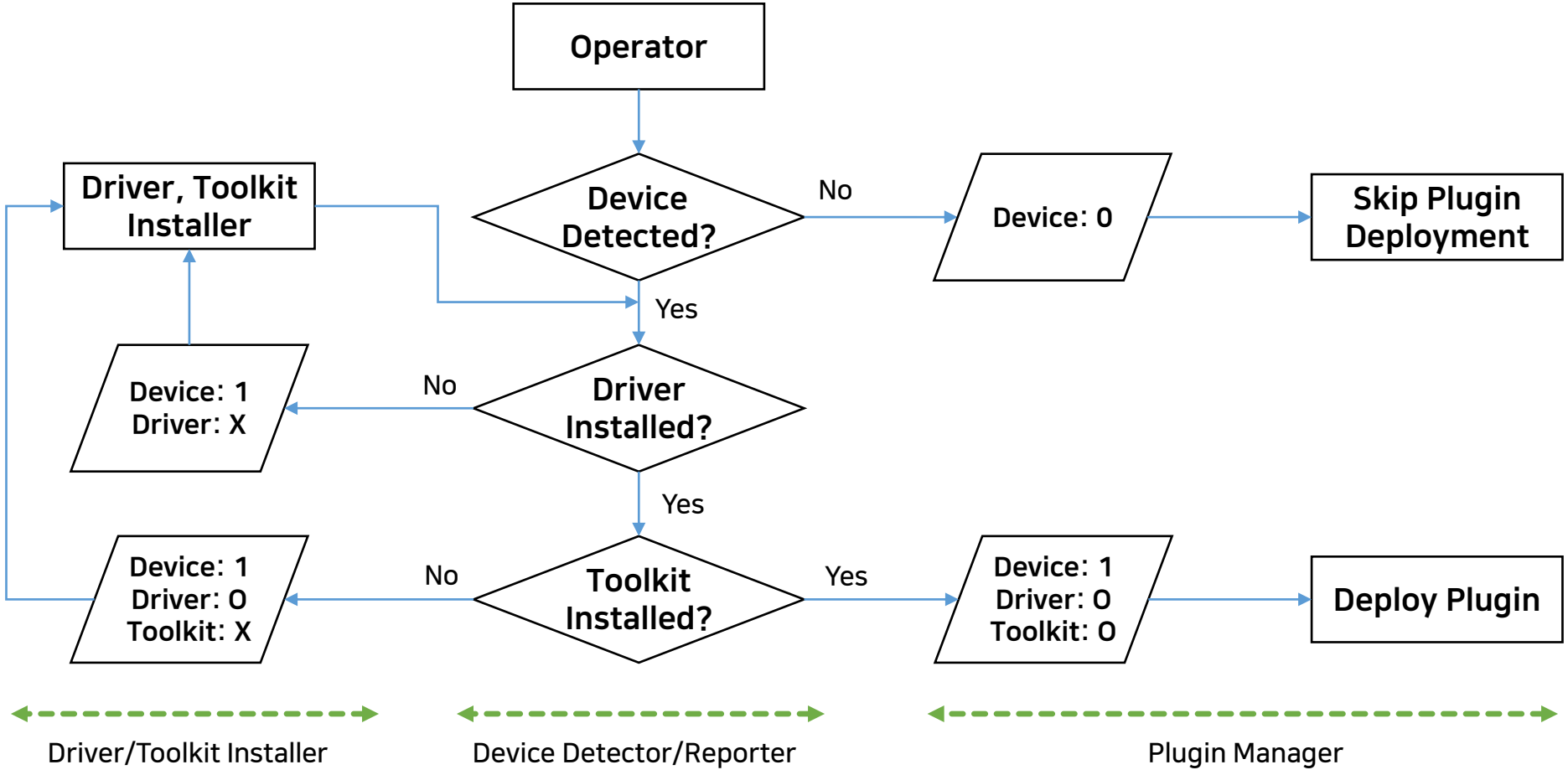
- 서로 다른 벤더, 가속기를 Kubernetes에 온보딩 하는 통합 Operator 필요

	직접 관리	이기종 가속기 통합 Operator
Node 설치 장치 확인	노드별 접근, 명령어 활용 확인	노드 장치 주기적 자동 Report
드라이버/툴킷 관리	노드별 설치, 업데이트	라벨 기반 자동 설치/업데이트
장치별 Plugin 배포/수정	Plugin YAML 직접 수정, 배포	CRD 값만 수정하면 자동 갱신, 배포
Node 증설/교체	새 노드에 모든 운영 장치 수동 설치	Operator가 자동 감지 후, 설치
멀티 벤더, 이기종 가속기	벤더, 모델별 설치 방법 관리 필요	Operator가 자동 관리
장애,오류 관리	개별 장치 리소스, 노드 확인 필요	Operator의 로그만 확인
장치/Plugin 삭제	장치, Plugin별 삭제	통합 CRD 삭제 시, 자동 제거

① 설치/운영의 반복 작업 최소화 → 오류 위험, 운영 비용 감소
② 운영 자동화 → 일관성, 재현성, 효율성 향상

이기종 가속기 통합 Operator (1/2)

- 개발목표 = PoC



이기종 가속기 통합 Operator (2/2)

- 수행절차

① 관리 자원 정의
- CRD(Custom Resource Definition)
- CR(Custom Resource)

② Kubernetes API (Spec) 정의

③ Controller 구현
- Reconcile Loop
- RBAC (Roll-Based Access Control)

④ Detector, Installer 구현

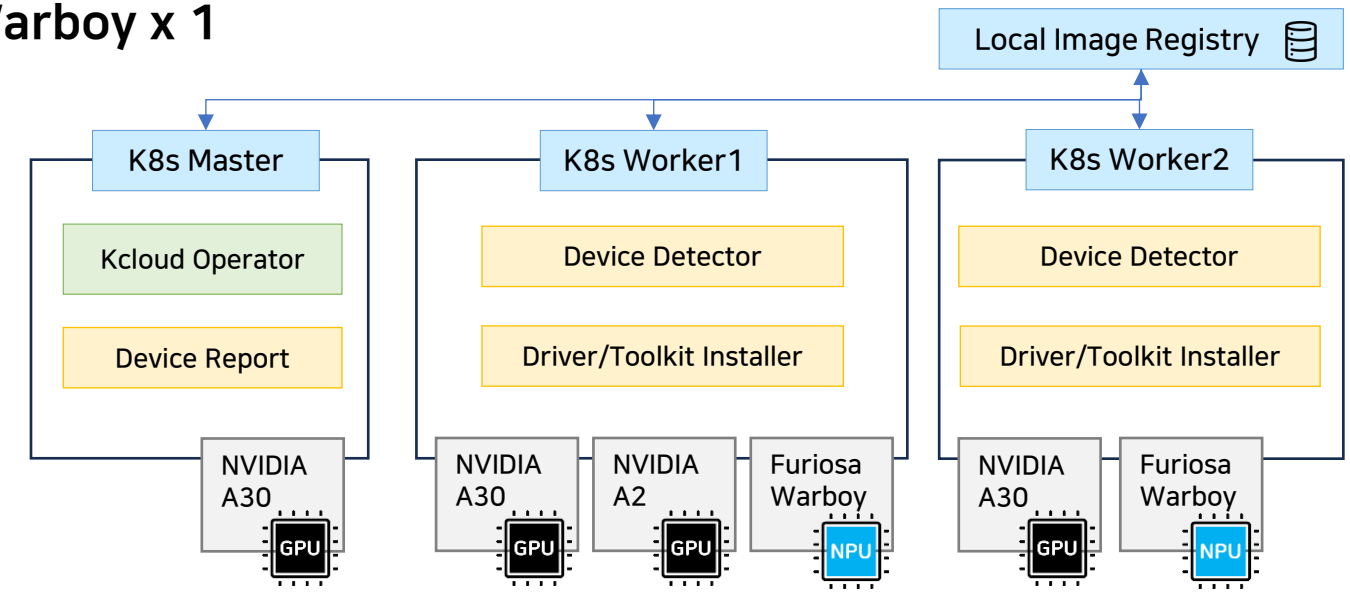
⑤ Container Image 구성
- Operator, Detector, Installer
- Local Repository 활용

⑤ Helm Chart
- CR/CRD/Role 설정 파일 포함
- 설치 자동화

<통합 Operator 개발 수행 절차>

시험환경

- Kubernetes 클러스터
  - Master 노드 1대
    - NVIDIA A30 x 1
  - Worker 노드 2대
    - Worker1: NVIDIA A30 x 1, NVIDIA A2 x 1, Furiosa Warboy x 1
    - Worker2: NVIDIA A30 x 1, Furiosa Warboy x 1
  - Ubuntu 22.04 (Kernel 5.15.0-161)
  - Kubernetes 1.28
  - Containerd 1.7.22
- Local Image Repository
  - Docker Registry



<Kubernetes 이종 가속기 연동 시험환경>

관리 자원 정의

- CRD, CR
  - NPUClusterPolicy
    - Device Plugin 배포
    - 클러스터 전체 적용
    - 벤더별 섹션 분리 → 추후 확장고려
  - DriverInstallPolicy
    - 커널, 런타임 버전 의존성 존재
    - 벤더 의존성 존재
    - → 별도 CRD 분리하여 관리
  - DeviceReport
    - HostPath 통해 노드 실제 상태 수집

```
spec.detector.image: Detector DS에서 사용할 이미지
spec.nvidia.enabled: NVIDIA Plugin 배포 여부
spec.nvidia.devicePluginImage: NVIDIA DP 이미지
spec.nvidia.nodeSelector: NVIDIA Plugin이 배포될 노드
spec.furiosa.enabled: Furiosa Plugin 배포 여부
spec.furiosa.devicePluginImage: Furiosa DP 이미지
spec.furiosa.configMapName: Furiosa configmap 참조 이름
spec.furiosa.nodeSelector: Furiosa Plugin 배포 대상 노드
...
```

<NPUClusterPolicy CRD 구조>

```
spec.vendor: 장치 vendor (nvidia, furiosa, rebellion.. 등)
spec.driver.image: 드라이버 설치 이미지
spec.driver.version: 설치할 드라이버 버전
spec.toolkit.enabled: toolkit 설치 여부
spec.toolkit.image: toolkit 설치 이미지
spec.kernelAllowlist: 허용되는 커널 버전
spec.containerdMinVersion: containerd 최소 버전
```

<DriverInstallPolicy CRD 구조>

```
status.devicePresent: 장치 존재 여부
status.driverInstalled: 드라이버 설치 여부
status.toolkitInstalled: 툴킷 설치 여부
status.vendor: 장치 벤더
status.model: 장치 모델
```

<DeviceReport CRD 구조>

```yaml
apiVersion: npu.ai/v1alpha1
kind: DriverInstallPolicy
metadata:
  name: nvidia-gpu
spec:
  vendor: nvidia
  model: generic
  rebootStrategy: IfNeeded
  kernelAllowlist: ["5.15.*", "6.8.*"]
  containerdMinVersion: "1.7.0"
  driver:
    installer: apt
    version: "nvidia-driver-535"
    image: 10.10.10.10:5100/nvidia-driver-installer:1.0
  toolkit:
    enabled: true
    method: apt
```

```yaml
apiVersion: npu.ai/v1alpha1
kind: NPUClusterPolicy
metadata:
  name: npuclusterpolicy-sample
spec:
  detector:
    image: 10.10.10.10:5100/npu-detector:1.0
  nvidia:
    enabled: true
    devicePluginImage: nvcr.io/nvidia/k8s-device-plugin:v0.17.1
    nodeSelector:
      nvidia.com/gpu.present: "true"
  furiosa:
    enabled: true
    devicePluginImage: ghcr.io/furiosa-ai/k8s-device-plugin:0.10.1
    configMapName: npu-device-plugin
    nodeSelector:
      furiosa: "true"
```

<Cluster/Install Policy CR 예>

Kubernetes API (Spec) 정의

- _types.go
  - Kubernetes API Server에 등록할 API 타입 정의
  - 해당 API를 기반으로
    - kubectl get, apply, watch 등이 동작
  - Operator에서 사용할 내부 데이터 모델
  - 관련 CRD yaml을 Kubebuilder로 자동 생성 시 활용
    - 앞서 설계한 CRD 구조를 따름

```go
package v1alpha1

import (
    metav1 "k8s.io/apimachinery/pkg/apis/meta/v1"
)

// +kubebuilder:object:root=true
// +kubebuilder:resource:scope=Cluster,shortName=ndr
// +kubebuilder:subresource:status
type NodeDeviceReport struct {
    metav1.TypeMeta   `json:",inline"`
    metav1.ObjectMeta `json:"metadata,omitempty"`

    Spec   NodeDeviceReportSpec   `json:"spec,omitempty"`
    Status NodeDeviceReportStatus `json:"status,omitempty"`
}

// ...

type DeviceEntry struct {
    Vendor        string `json:"vendor,omitempty"` // "rebellion" | furiosa" | "nvidia" 등
    Model         string `json:"model,omitempty"`  // "warboy" 등
    Count         int32  `json:"count,omitempty"`
    DriverLoaded  bool   `json:"driverLoaded,omitempty"`
    DriverVersion string `json:"driverVersion,omitempty"`
    NeedsReboot   bool   `json:"needsReboot,omitempty"`
}

type Condition struct {
    Type    string `json:"type,omitempty"`
    Status  string `json:"status,omitempty"` // "True"|"False"|"Unknown"
    Reason  string `json:"reason,omitempty"`
    Message string `json:"message,omitempty"`
}

// ...
```

<nodedevicereport_types.go 구조>

Controller 구현 (1/3)

- _controller.go
  - 관련 CRD/CR에 대한 관찰/분석/실행(생성/업데이트/삭제) 로직
  - NPUClusterPolicy Reconciler
    - Detector DaemonSet 실행 보장
    - Device Plugin DS 자동 생성
    - Vendor별 ConfigMap 자동 생성/업데이트
    - nodeSelector 기반 선택적 배포

```go
func (r *NPUClusterPolicyReconciler) Reconcile(ctx context.Context, req ctrl.Request) (ctrl.Result, error) {
    logger := logf.FromContext(ctx)
    logger.Info("Reconciling NPUClusterPolicy", "name", req.NamespacedName)

    //-- Get CR
    var policy npuv1alpha1.NPUClusterPolicy
    if err := r.Get(ctx, req.NamespacedName, &policy); err != nil {
        logger.Error(err, "unable to fetch NPUClusterPolicy")
        return ctrl.Result{}, client.IgnoreNotFound(err)
    }
    // -- Detector
    if err := r.ensureDetector(ctx, &policy); err != nil {
        logger.Error(err, "failed to ensure Detector")
        return ctrl.Result{}, err
    }
    //-- NVIDIA
    if policy.Spec.Nvidia.Enabled {
        logger.Info("Ensuring NVIDIA Device Plugin DaemonSet")
        if err := r.ensureNvidiaDevicePlugin(ctx, &policy); err != nil {
            logger.Error(err, "failed to ensure NVIDIA Device Plugin")
            return ctrl.Result{}, err
        }
    }
    //-- Furiosa
    if policy.Spec.Furiosa.Enabled {
        logger.Info("Ensuring Furiosa Device Plugin DaemonSet")
        if err := r.ensureFuriosaDevicePlugin(ctx, &policy); err != nil {
            logger.Error(err, "failed to ensure Furiosa Device Plugin")
            return ctrl.Result{}, err
        }
    }
    return ctrl.Result{}, nil
}
```

<npuclusterpolicy_controller.go 구조>

Controller 구현 (2/3)

- _controller.go
  - 관련 CRD/CR에 대한 관찰/분석/실행(생성/업데이트/삭제) 로직
  - DriverInstaller Reconciler
    - NodeDeviceReport 기반 설치 필요 판단
    - Installer Job 생성/관리
    - 설치 완료 후 node label 업데이트
    - Job 재시도/쿨다운 처리
    - HostPath mount, secret mount 등 설치환경 자동 구성

```go
func (r *DriverInstallReconciler) Reconcile(ctx context.Context, req ctrl.Request) (ctrl.Result, error) {
    var rep npuv1alpha1.NodeDeviceReport
    if err := r.Get(ctx, req.NamespacedName, &rep); err != nil {
        return ctrl.Result{}, client.IgnoreNotFound(err)
    }
    // If the driver is already installed, apply node labels and exit
    if isDriverLoaded(rep) {
        r.ensureNodeLabels(ctx, &rep)
        return ctrl.Result{}, nil
    }
    // Fetch all DriverInstallPolicy objects
    var pols npuv1alpha1.DriverInstallPolicyList
    if err := r.List(ctx, &pols); err != nil {
        return ctrl.Result{}, err
    }
    // Find the matching DriverInstallPolicy for this node/device
    policy := matchPolicy(&rep, pols)
    if policy == nil {
        return ctrl.Result{}, nil
    }
    // Create an install Job based on the matched policy
    job := r.renderInstallJob(rep.Spec.NodeName, policy, &rep)
    if err := r.Create(ctx, &job); err != nil {
        return ctrl.Result{}, err
    }
    return ctrl.Result{RequeueAfter: time.Minute}, nil
}
```

<dirverinstall_controller.go 구조>

Controller 구현 (3/3)

- Kubernetes 제어 권한 설정
  - Role (제어 권한 정의)
  - RoleBinding (그 권한을 누가 쓸 것인지)
  - ServiceAccount (Pod가 API Server 요청 시, 사용하는 인증)

pod

ServiceAccount

```yaml
apiVersion: v1
kind: ServiceAccount
metadata:
  name: controller-manager
  namespace: system
  labels:
    app.kubernetes.io/name: npu-operator
    app.kubernetes.io/managed-by: kustomize
```

RoleBinding

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  labels:
    app.kubernetes.io/name: npu-operator
    app.kubernetes.io/managed-by: kustomize
  name: manager-rolebinding
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: manager-role
subjects:
- kind: ServiceAccount
  name: controller-manager
  namespace: system
```

Role

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: manager-role
rules:
- apiGroups: [""]
  resources: ["configmaps"]
  verbs: ["create", "delete", "get", "list", "patch", "update", "watch"]

- apiGroups: ["apps"]
  resources: ["daemonsets"]
  verbs: ["create", "delete", "get", "list", "patch", "update", "watch"]

- apiGroups: ["npu.ai"]
  resources: ["npuclusterpolicies"]
  verbs: ["create", "delete", "get", "list", "patch", "update", "watch"]
```

Resource

NPUClusterPolicy

DriverInstallPolicy

ds

cm

⋮

<Operator의 Kubernetes 제어 권한 설정>

Detector, Installer 구현 (1/2)

- Operator에 포함되지 않는 독립 컨테이너(이미지)화
  - 노드 내부(Host OS)에서 직접 실행되는 OS 레벨 작업
    - Operator는 Kubernetes Control Plane 실행, 개별 노드 권한 없음
    - Kubelet은 CPU, 메모리 정보는 확인 가능 → 이기종 가속기는? 확인 불가
  - 벤더별 맞춤 로직 구현 필요
    - PCI 장치, 장치 드라이버, Toolkit 설치 위치, 버전.. 등

- Detector
  - 노드의 (/sys, /dev) .. 읽기
  - PCI 디바이스 확인
  - 커널 모듈 확인 (/sys/module)
  - 디바이스 파일 확인 등

  → Host OS 파일 시스템 접근 필요

- Installer
  - 커널 모듈 설치
  - dkms build
  - apt/dpkg 설치
  - /lib/modules 수정 등

  → Host OS 파일 시스템 접근 필요

Detector, Installer 구현 (2/2)

- Detector와 Operator 연동
  - Operator의 CR인 NodeDeviceReport를 지속적 업데이트

```go
gvk := schema.GroupVersion{Group: "npu.ai", Version: "v1alpha1"}
gk := schema.GroupVersionKind{Group: gvk.Group, Version: gvk.Version, Kind: "NodeDeviceReport"}

for {
    dets := detect()
    devices := make([]map[string]any, 0, len(dets))
    for _, d := range dets {
        devices = append(devices, map[string]any{
            "vendor":              d.vendor,
            "model":               d.model,
            "count":               int32(d.count),
            "driverLoaded":        d.loaded,
            "driverVersion":       d.ver,
            "driverVersionDetail": d.detail, )
        })
    }

    ndr := &unstructured.Unstructured{}
    ndr.SetGroupVersionKind(gk)
    ndr.SetName(node)

    ctx := context.Background()
    // Get or Create
    if err := c.Get(ctx, client.ObjectKey{Name: node}, ndr); err != nil {
        ndr.Object = map[string]any{
            "apiVersion": gvk.String(),
            "kind":       "NodeDeviceReport",
            "metadata":   map[string]any{"name": node},
            "spec":       map[string]any{"nodeName": node},
            "status":     map[string]any{"devices": devices},
        }
```

```go
        if err := c.Create(ctx, ndr); err != nil {
            fmt.Println("create NDR err:", err)
        } else {
            _ = c.Status().Update(ctx, ndr)
            fmt.Println("NDR created for", node)
        }
    } else {
        if ndr.Object["status"] == nil {
            ndr.Object["status"] = map[string]any{}
        }
        ndr.Object["status"].(map[string]any)["devices"] = devices
        if err := c.Status().Update(ctx, ndr); err != nil {
            if ndr.Object["spec"] == nil {
                ndr.Object["spec"] = map[string]any{"nodeName": node}
            }
            _ = c.Update(ctx, ndr)
        }
        fmt.Println("NDR updated for", node, "devices:", len(devices))
    }
    time.Sleep(30 * time.Second)
}
```

<Detector의 Operator 연동 함수>

Container Image 및 Helm Chart 구성

- Container Image
  - Kubernetes 클러스터 배포 후, 실행 컨테이너 생성을 위한 이미지화
    - docker build/push ..

```
kcloud@k8s-master:~$ curl -s "http://10.10.10.10:5100/v2/_catalog?n=1000" \
| jq -r '.repositories[]' \
| while read repo; do
    curl -s "http://10.10.10.10:5100/v2/${repo}/tags/list" \
    | jq -r --arg repo "$repo" \
        '.tags // [] | .[] | "\($repo):\(. )"'
  done

furiosa-driver-installer:1.0
npu-detector:1.0
npu-operator:1.0
nvidia-driver-installer:1.0
```

<Operator 관련 생성 이미지 예>

- Helm Chart
  - Operator 동작에 필요한 리소스 배포 자동화
    - 기존 리소스 Helm Value 기반 매개변수화 필요
  - 버전 관리
  - 벤더별 옵션 변경

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  labels:
    app.kubernetes.io/name: npu-operator
    app.kubernetes.io/managed-by: kustomize
  name: manager-rolebinding
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: manager-role
subjects:
```

→

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: {{ include "npu-operator.fullname" . }}-rolebinding
  labels:
    app.kubernetes.io/name: {{ include "npu-operator.name" . }}
    app.kubernetes.io/instance: {{ .Release.Name }}
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: {{ include "npu-operator.fullname" . }}-role
subjects:
```

<Operator 관련 Helm Value 기반 매개변수화 예>

Kubernetes에서 이기종 가속기의 컨테이너 연동 (1/9)

- Operator 배포 확인

```
kcloud@k8s-master:~/operator/npu-operator/npu-operator$ make deploy IMG=$IMG
/home/kcloud/operator/npu-operator/npu-operator/bin/controller-gen rbac:roleName=manager-role crd webhook paths="./..." output:crd:artifacts:co
cd config/manager && /home/kcloud/operator/npu-operator/npu-operator/bin/kustomize edit set image controller=10.10.10.10:5100/npu-operator:1.0
/home/kcloud/operator/npu-operator/npu-operator/bin/kustomize build config/default | kubectl apply -f -
namespace/npu-operator-system created
customresourcedefinition.apiextensions.k8s.io/driverinstallpolicies.npu.ai created
customresourcedefinition.apiextensions.k8s.io/nodedevicereports.npu.ai created
customresourcedefinition.apiextensions.k8s.io/npuclusterpolicies.npu.ai created
serviceaccount/npu-operator-controller-manager created
serviceaccount/npu-operator-npu-detector created
role.rbac.authorization.k8s.io/npu-operator-leader-election-role created
clusterrole.rbac.authorization.k8s.io/npu-operator-manager-role created
clusterrole.rbac.authorization.k8s.io/npu-operator-metrics-auth-role created
clusterrole.rbac.authorization.k8s.io/npu-operator-metrics-reader created
clusterrole.rbac.authorization.k8s.io/npu-operator-npu-detector created
clusterrole.rbac.authorization.k8s.io/npu-operator-npuclusterpolicy-admin-role created
clusterrole.rbac.authorization.k8s.io/npu-operator-npuclusterpolicy-editor-role created
clusterrole.rbac.authorization.k8s.io/npu-operator-npuclusterpolicy-viewer-role created
rolebinding.rbac.authorization.k8s.io/npu-operator-leader-election-rolebinding created
clusterrolebinding.rbac.authorization.k8s.io/npu-operator-manager-rolebinding created
clusterrolebinding.rbac.authorization.k8s.io/npu-operator-metrics-auth-rolebinding created
clusterrolebinding.rbac.authorization.k8s.io/npu-operator-npu-detector created
service/npu-operator-controller-manager-metrics-service created
deployment.apps/npu-operator-controller-manager created
```

<Operator 배포 및 생성 Pod 확인>

```
kcloud@k8s-master:~$ kubectl -n npu-operator-system get pod
NAME                                          READY   STATUS    RESTARTS   AGE
npu-operator-controller-manager-6b68d98d48-qh5c7   1/1     Running   0          32s
```

Kubernetes에서 이기종 가속기의 컨테이너 연동 (2/9)

- Detector 동작 확인
  - Worker 노드별 개별 Pod로 실행되어 시스템 확인

```
kcloud@k8s-master:~$ kubectl get ds -A | grep -E '^NAMESPACE|npu-'
NAMESPACE     NAME                  DESIRED   CURRENT   READY   UP-TO-DATE   AVAILABLE   NODE SELECTOR                AGE
kube-system   npu-detector          3         3         3       3            3           <none>                       47h

kcloud@k8s-master:~$ kubectl get pods -A -o wide | awk 'NR==1 || $2 ~ /^npu-/' | awk '{print $1,$2,$3,$4,$6,$7}' | column -t
NAMESPACE            NAME                                          READY  STATUS   AGE    IP
kube-system          npu-detector-dxd2q                            1/1    Running  7m40s  10.233.92.134
kube-system          npu-detector-v8gqz                            1/1    Running  7m40s  10.233.123.69
npu-operator-system  npu-operator-controller-manager-6b68d98d48-qh5c7  1/1    Running  17m    10.233.92.133
```

<npu-detector 실행 결과>

Kubernetes에서 이기종 가속기의 컨테이너 연동 (3/9)

- Detector 동작 확인
  - K8s-worker1 기준
    - GPU X 2개, NPU X 1개
    - 장치 드라이버 설치 GPU O, NPU X

```
kcloud@k8s-master:~$ kubectl get nodedevicereports.npu.ai k8s-worker1 -o yaml
- apiVersion: npu.ai/v1alpha1
  kind: NodeDeviceReport
  metadata:
    creationTimestamp: "2025-11-16T09:10:04Z"
    generation: 1
    name: k8s-worker1
    resourceVersion: "182120"
    uid: d6e42bd7-9527-45e3-928a-ba8cf429c3ef
  spec:
    nodeName: k8s-worker1
  status:
    devices:
    - count: 1
      driverLoaded: false
      driverVersion: ""
      model: warboy
      vendor: furiosa
    - count: 2
      driverLoaded: true
      driverVersion: 580.95.05
      model: generic
      vendor: nvidia
```

<nodedevicereports 출력>

```
kcloud@k8s-worker1:~$ lspci -v |grep NVIDIA
18:00.0 3D controller: NVIDIA Corporation GA100GL [A30 PCIe] (rev a1)
        Subsystem: NVIDIA Corporation GA100GL [A30 PCIe]
86:00.0 3D controller: NVIDIA Corporation GA107GL [A2 / A16] (rev a1)
        Subsystem: NVIDIA Corporation Device 157e
kcloud@k8s-worker1:~$
kcloud@k8s-worker1:~$ nvidia-smi
Sun Nov 16 18:23:13 2025
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 580.95.05              Driver Version: 580.95.05      CUDA Version: 13.0     |
+-----------------------------------------+------------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================|
|   0  NVIDIA A30                     Off |   00000000:18:00.0 Off |                    0 |
| N/A   33C    P0             26W /  165W |       0MiB /  24576MiB |      0%      Default |
|                                         |                        |             Disabled |
+-----------------------------------------+------------------------+----------------------+
|   1  NVIDIA A2                      Off |   00000000:86:00.0 Off |                    0 |
|  0%   33C    P8              5W /   60W |       0MiB /  15356MiB |      0%      Default |
|                                         |                        |                  N/A |
+-----------------------------------------+------------------------+----------------------+

+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
|  GPU   GI   CI              PID   Ty
|        ID   ID
|=========================================
|  No running processes found
+-----------------------------------------
```

```
kcloud@k8s-worker1:~$ lspci -v |grep Furiosa
af:00.0 Processing accelerators: FuriosaAI, Inc. Warboy (rev 01)
        Subsystem: FuriosaAI, Inc. Warboy
kcloud@k8s-worker1:~$
kcloud@k8s-worker1:~$ furiosactl info
furiosactl: command not found
```

<K8s-worker1 노드의 장치 및 장치 드라이버 상태>

Kubernetes에서 이기종 가속기의 컨테이너 연동 (4/9)

- Installer 동작 확인
  - K8s-worker1의 NPU 장치 드라이버 및 컨테이너 툴킷 설치
  - 30초 마다 nodedevicereports 확인 후, 설치 X일 때 Pod 생성하여 실행, 완료 시 Complete
  - 라벨 기반(npu.driver.install=true)

```
kcloud@k8s-master:~/operator/npu-operator/npu-operator$ kubectl label node k8s-worker1 npu.driver.install=true --overwrite
node/k8s-worker1 labeled

kcloud@k8s-master:~$ kubectl get node -o wide --show-labels |grep .install
k8s-worker1   Ready    <none>          31h   v1.28.0   129.254.202.241   <none>        Ubuntu 22.04.3 LTS   5.15.0-161-generic   containerd://1.7.22
beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,kubernetes.io/arch=amd64,kubernetes.io/hostname=k8s-worker1,kubernetes.io/os=linux,
npu.driver.install=true

kcloud@k8s-master:~$ kubectl get pods -A | awk 'NR==1 || $2 ~ /^npu-driver/' | awk '{print $1,$2,$3,$4,$6,$7}' | column -t
NAMESPACE    NAME                                READY  STATUS     AGE
kube-system  npu-driverinstall-k8s-worker1-p4cs2  0/1    Completed  3m1s
```

<npu-driverinstaller 생성 및 실행 완료 확인>

Kubernetes에서 이기종 가속기의 컨테이너 연동 (5/9)

- **Installer 동작 확인**

```
kcloud@k8s-master:~$ kubectl -n kube-system logs npu-driverinstall-k8s-worker1-p4cs2
[INFO] Preparing host APT directories...
[INFO] Fetching Furiosa public key (container side)...
[INFO] Installing APT key/auth/list to host...
[INFO] Waiting for APT locks & pre-fix...
[INFO] Installing Furiosa packages on host...
Get:1 https://archive.furiosa.ai/ubuntu jammy InRelease [4302 B]
```

```
NEEDRESTART-SVC: user@1000.service
furiosa-driver-warboy set on hold.
furiosa-libhal-warboy set on hold.
furiosa-libnux set on hold.
[INFO] Done. Host packages installed.
```

<npu-driverinstaller 실행 로그(NPU, Furiosa)>

```
kcloud@k8s-worker1:~$ furiosactl info
+------+--------+----------------+-------+---------+--------------+
| NPU  | Name   | Firmware       | Temp. | Power   | PCI-BDF      |
+------+--------+----------------+-------+---------+--------------+
| npu0 | warboy | 1.7.7, 386a8ab |  45°C | 10.28 W | 0000:af:00.0 |
+------+--------+----------------+-------+---------+--------------+
```

<K8s-worker1에서 NPU, Furiosa 장치 드라이버 설치 확인>

```
kcloud@k8s-master:~$ kubectl get nodedevicereports.npu.ai k8s-worker1 -o yaml
- apiVersion: npu.ai/v1alpha1
  kind: NodeDeviceReport
  metadata:
    creationTimestamp: "2025-11-16T09:10:04Z"
    generation: 1
    name: k8s-worker1
    resourceVersion: "184145"
    uid: d6e42bd7-9527-45e3-928a-ba8cf429c3ef
  spec:
    nodeName: k8s-worker1
  status:
    devices:
    - count: 1
      driverLoaded: true
      driverVersion: 1.9.9-3
      model: warboy
      vendor: furiosa
    - count: 2
      driverLoaded: true
      driverVersion: 580.95.05
      model: generic
      vendor: nvidia
```

< nodedevicereports 출력 재확인>

Kubernetes에서 이기종 가속기의 컨테이너 연동 (6/9)

- **Installer 동작 확인**

```
kcloud@k8s-master:~$ kubectl -n kube-system logs npu-driverinstall-k8s-worker2-8lrds | head -n 30
[INFO] Starting NVIDIA driver & toolkit installation
[INFO] CURRENT_KERNEL=5.15.0-161-generic
[INFO] Pre-install cleanup: dpkg state, locks, crash reports
[INFO] Apt update + base tools
Hit:1 https://archive.furiosa.ai/ubuntu jammy InRelease
```

```
time="2025-11-16T23:45:26+09:00" level=info msg="Generated CDI spec with version 1.0.0"
[INFO] nouveau already blacklisted
[INFO] Runtime already configured
[INFO] NVIDIA driver and toolkit installation completed successfully.
```

<npu-driverinstaller 실행 로그(GPU, Nvidia>

```
kcloud@k8s-worker2:~$ hostname
k8s-worker2

kcloud@k8s-worker2:~$ nvidia-smi
Sun Nov 16 23:55:03 2025
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 535.274.02             Driver Version: 535.274.02   CUDA Version: 12.2     |
|-----------------------------------------+----------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id        Disp.A | Volatile Uncorr. ECC |
```

<K8s-worker2에서 GPU, Nvidia 장치 드라이버 설치 확인>

```
kcloud@k8s-master:~$ kubectl get nodedevicereports.npu.ai k8s-worker2 -o yaml
apiVersion: v1
items:
- apiVersion: npu.ai/v1alpha1
  kind: NodeDeviceReport
  metadata:
    creationTimestamp: "2025-11-16T14:09:28Z"
    generation: 1
    name: k8s-worker2
    resourceVersion: "216425"
    uid: 25ddb37b-49b8-4120-8905-686
  spec:
    nodeName: k8s-worker2
  status:
    devices:
    - count: 1
      driverLoaded: true
      driverVersion: 1.9.9-3
      model: warboy
      vendor: furiosa
    - count: 1
      driverLoaded: false
      driverVersion: ""
      model: generic
      vendor: nvidia
```

```
kcloud@k8s-master:~$ kubectl get nodedevicereports.npu.ai k8s-worker2 -o yaml
apiVersion: v1
items:
- apiVersion: npu.ai/v1alpha1
  kind: NodeDeviceReport
  metadata:
    creationTimestamp: "2025-11-16T14:12:18Z"
    generation: 1
    name: k8s-worker2
    resourceVersion: "216425"
    uid: 25ddb37b-49b8-4120-8905-686f1ad098ee
  spec:
    nodeName: k8s-worker2
  status:
    devices:
    - count: 1
      driverLoaded: true
      driverVersion: 1.9.9-3
      model: warboy
      vendor: furiosa
    - count: 1
      driverLoaded: true
      driverVersion: 535.274.02
      model: generic
      vendor: nvidia
```

< nodedevicereports 출력 확인>

Kubernetes에서 이기종 가속기의 컨테이너 연동 (7/9)

- Device Plugin 배포 확인
  - 라벨 기반 해당 벤더 디바이스 플러그인 배포

```
kcloud@k8s-master:~$ kubectl label node k8s-worker1 furiosa=true --overwrite
kcloud@k8s-master:~$ kubectl label node k8s-worker1 nvidia.com/gpu.present=true --overwrite

kcloud@k8s-master:~$ kubectl get node -o wide --show-labels
k8s-worker1   Ready    <none>          3d12h   v1.28.0   129.254.202.241   <none>        Ubuntu 22.04.3 LTS   5.15.0-161-generic   containerd://1.7.22
beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,furiosa=true,kubernetes.io/arch=amd64,kubernetes.io/hostname=k8s-worker1,kubernetes.io/os=linux,
npu.driver.install=true

kcloud@k8s-master:~$ kubectl get pods -A | awk 'NR==1 || $2 ~ /^(furiosa|nvidia)/' | awk '{print $1,$2,$3,$4,$6,$7}' | column -t
NAMESPACE    NAME                        READY  STATUS   AGE
kube-system  furiosa-device-plugin-8l6vp  1/1    Running  46h
kube-system  nvidia-device-plugin-h6kq9   1/1    Running  46h
```

<NPU, Furiosa Device Plugin DS 생성 및 실행 결과>

Kubernetes에서 이기종 가속기의 컨테이너 연동 (8/9)

- Pod에 이기종 가속기 할당 확인

```
kcloud@k8s-master:~/operator/pods$ cat gpu-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: gpu-pod
spec:
  containers:
  - name: cuda
    image: nvidia/cuda:12.9.1-base-ubuntu22.04
    command: [ "sleep", "infinity" ]
    resources:
      limits:
        nvidia.com/gpu: 1
```

```
kcloud@k8s-master:~$ kubectl get pods -A -o wide| awk 'NR==1 || $2 ~ /
NAMESPACE   NAME     READY   STATUS    AGE   IP
default     gpu-pod  1/1     Running   21s   10.233.118.147
```

<GPU Pod 생성>

```
kcloud@k8s-master:~/operator/pods$ cat npu-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: npu-pod
spec:
  containers:
  - name: npu-pod
    image: docker.io/khdbsfdk/furiosa_ai_sdk:v0.1
    command: ["/bin/bash"]
    args: ["-c", "rm -rf /etc/apt/sources.list.d/furiosa.list /etc/apt/keyrings/furiosa-apt-key.gpg;
    ls /etc/apt/keyrings/; while true; do echo hello; sleep 10;done"]
    resources:
      limits:
        cpu: "4"
        memory: "8Gi"
        beta.furiosa.ai/npu: "1"
      requests:
        cpu: "4"
        memory: "8Gi"
        beta.furiosa.ai/npu: "1"
```

```
kcloud@k8s-master:~/operator/pods$ kubectl get pods -A -o wide| awk 'NR==1 || $2 ~ /^npu-pod/' |
NAMESPACE   NAME     READY   STATUS    AGE    IP
default     npu-pod  1/1     Running   3m34s  10.233.92.140
```

<NPU Pod 생성>

Kubernetes에서 이기종 가속기의 컨테이너 연동 (9/9)

- Pod에 이기종 가속기 할당 확인

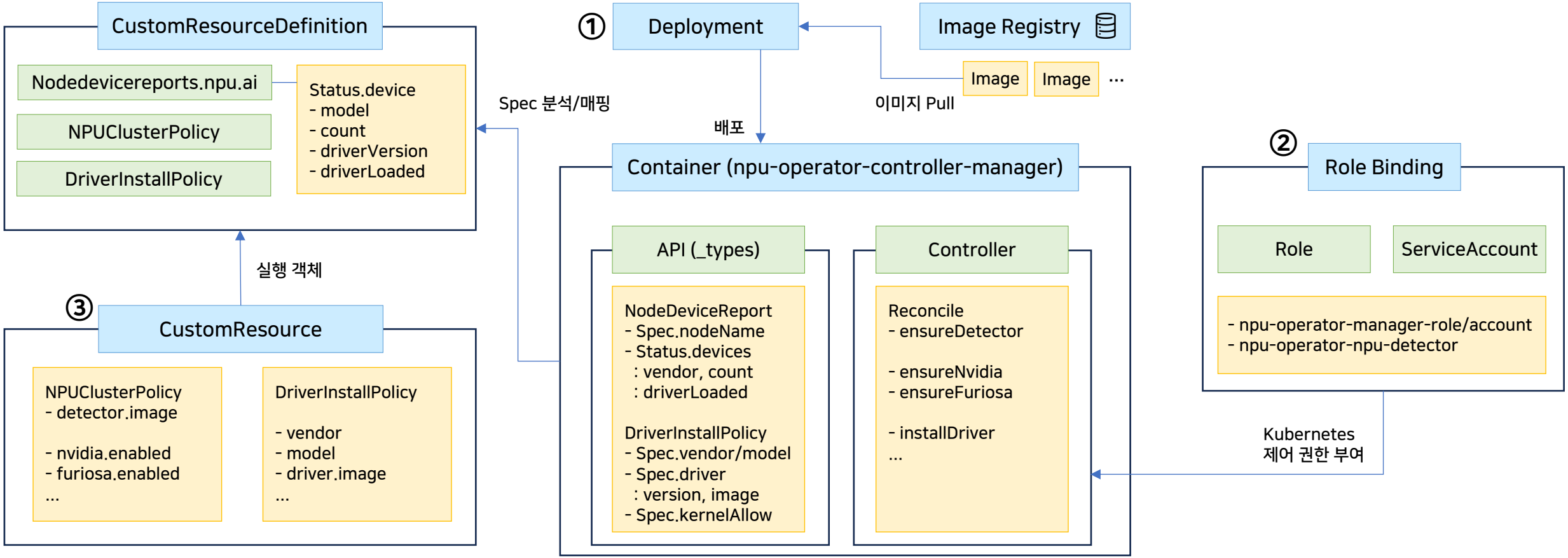
```
kcloud@k8s-master:~$ kubectl exec -it gpu-pod -- /bin/bash
root@gpu-pod:/#
root@gpu-pod:/# nvidia-smi
Sun Nov 16 15:49:46 2025
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 535.274.02             Driver Version: 535.274.02   CUDA Version: 12.9     |
|-----------------------------------------+----------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |         Memory-Usage | GPU-Util  Compute M. |
|                                         |                      |               MIG M. |
|=========================================+======================+======================|
|   0  NVIDIA A30                     Off | 00000000:18:00.0 Off |                    0 |
| N/A   36C    P0              30W / 165W |      0MiB / 24576MiB |      0%      Default |
|                                         |                      |             Disabled |
+-----------------------------------------+----------------------+----------------------+

+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
|  GPU   GI   CI        PID   Type   Process name                            GPU Memory |
|        ID   ID                                                             Usage      |
|=========================================================================================|
|  No running processes found                                                             |
+-----------------------------------------------------------------------------------------+
```

<GPU Pod 장치 할당 확인>

```
kcloud@k8s-master:~/operator/pods$ kubectl exec -it npu-pod -- /bin/bash
root@npu-pod:~#
root@npu-pod:~# furiosactl info
+------+--------+----------------+-------+--------+--------------+
| NPU  | Name   | Firmware       | Temp. | Power  | PCI-BDF      |
+------+--------+----------------+-------+--------+--------------+
| npu0 | warboy | 1.7.7, 386a8ab |  45°C | 9.86 W | 0000:af:00.0 |
+------+--------+----------------+-------+--------+--------------+
```

<NPU Pod 장치 할당 확인>

이기종 가속기 통합 Operator 구조



<이기종 가속기 통합 Operator 구조>

요약 및 향후 계획

- **요약**

① 서로 다른 벤더, 가속기를 Kubernetes에서?

- Kubernetes Plugin..
- 설치, 배포, 장애 추적 등 운영 업무 및 난이도 증가 ↑

→

② Kubernetes의 Controller 개념 활용한 Operator 구성

- API 설계 (CRD, CR)
- Reconcile Loop 구현
- RBAC 설정
- 별도 유틸 연동(Detector, Installer)

→

③ PoC 검증

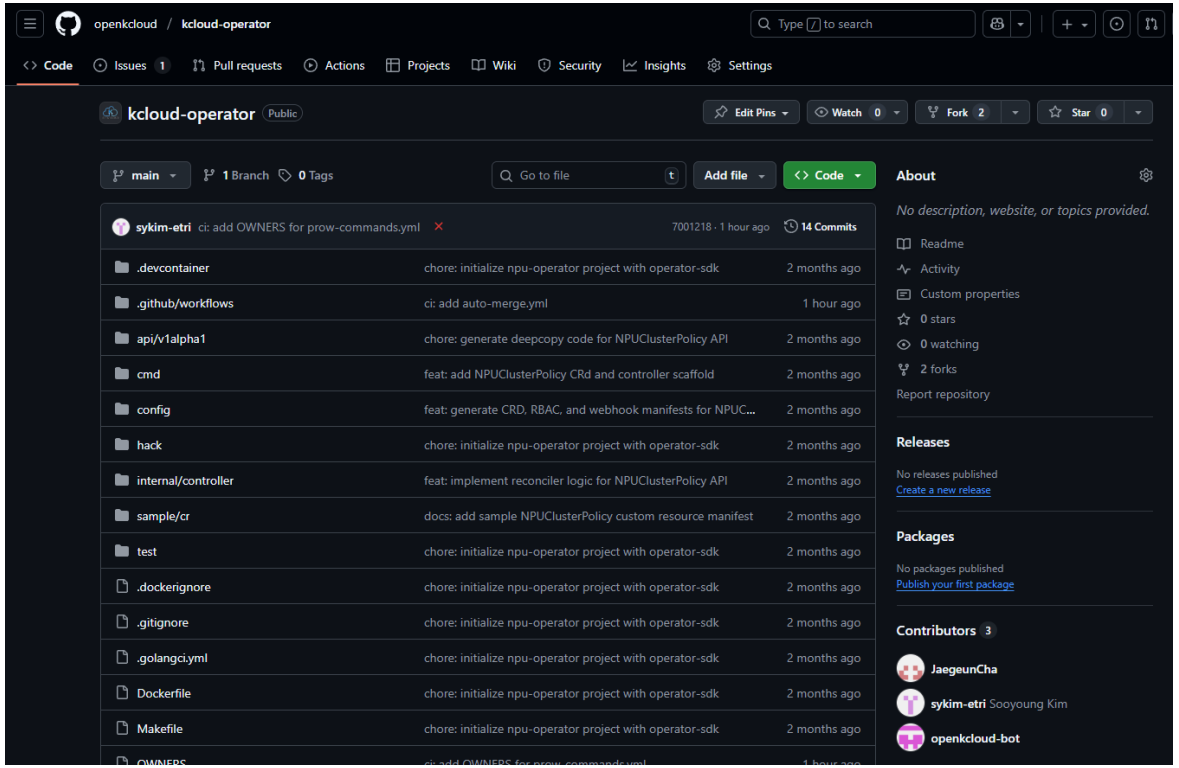
- NVIDIA GPU, Furiosa Warboy
- 드라이버/툴킷 자동 설치
- 플러그인 자동 배포

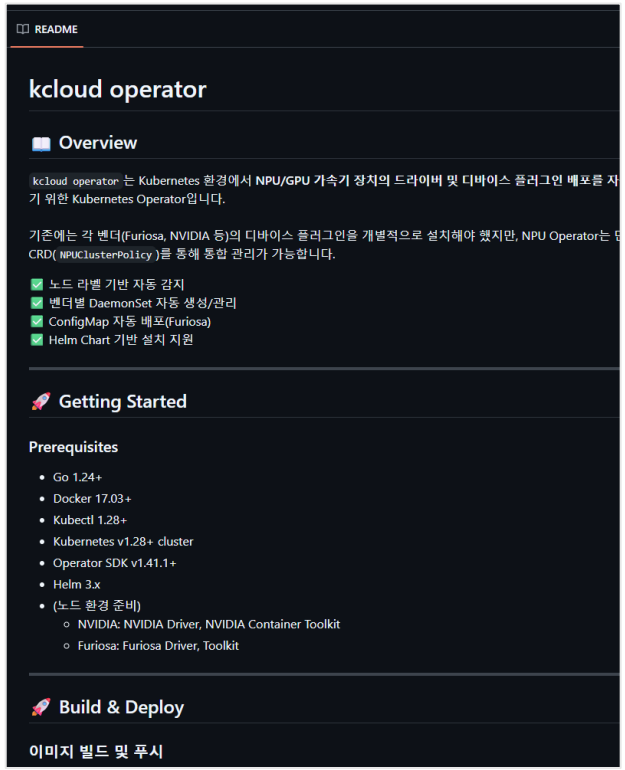
- **향후 계획 (기능 확장)**

① 신규 NPU 적용, 기능 추가 자동화
② 관리 모듈 추가 (장치 상태 모니터링, Node Feature Discovery)
③오류 추적 및 오류 발생 시, 자동 롤백
④ …

결과물 Github Repository

- Openkcloud/kcloud-operator
  - https://github.com/openkcloud/kcloud-operator





< kcloud github의 operator repository >

감사합니다.